计算结构力学课程大作业

平面桁架直接刚度法计算报告

必做题与自选题的手算、机算及程序实现

学 院（系）：_____

专　　业：_____

组 长 姓 名：_____

组 员 姓 名：_____

完 成 日 期：_____

大连理工大学

Dalian University of Technology

目　录

1 计算任务与模型约定.....1
1.1 任务内容与文件来源.....1
1.2 基本假设、坐标与单位.....1
1.3 计算与复核路线.....2
2 直接刚度法与程序算法.....3
2.1 单元几何及刚度矩阵.....3
2.2 总刚组装与支座处理.....3
2.3 方程求解、内力恢复与校核.....3
3 必做题：四节点六杆桁架.....5
3.1 结构数字化与荷载分解.....5
3.2 手算：缩减刚度方程与位移.....6
3.3 手算：杆件轴力与支反力.....7
3.4 机算结果及手算对照.....8
4 选做题：五节点七杆桁架.....9
4.1 自选模型与数字化.....9
4.2 手算：整体平衡与节点法.....10
4.3 手算：杆伸长与全部自由位移.....11
4.4 机算刚度方程与结果对照.....12
5 程序功能、使用与计算结论.....14
5.1 功能与数据组织.....14
5.2 运行方式与输出内容.....14
5.3 实际校核与软件特点.....15
5.4 计算结论.....15
附录 A 程序全文和程序注释.....16
附录 B 必做题与选做题总控信息.....30
B.1 必做题总控信息.....30
B.2 选做题总控信息.....30

1 计算任务与模型约定

1.1 任务内容与文件来源

本报告采用直接刚度法完成平面桁架静力计算，交付计算报告、带详细注释的 C 语言程序，以及必做题和选做题两份总控信息文本。每题均给出可复核的手算过程、全部可动节点位移及全部杆件轴力，并通过节点平衡、整体平衡和能量关系验证机算结果。

必做题的连接关系、支座条件、荷载、材料及截面取自用户上传仓库 xiaoxiaoshengwu/structure-work 中的“必做题_总控信息.txt”。本次使用提交 e6b174236a374e92f8f06a51fcc683aec7623158 的文件。根据用户确认，数值计算直接采用总控信息中的 l=h=1 m；原题示意图未给定数值尺寸，故这两个值属于本报告的补充计算假定。为便于替换尺寸，第 3 章同时给出一般 l、h 的解析表达式。

选做题自选为五节点七杆静定桁架，采用 8 m 跨度、1.5 m 上节点高度、不同荷载及单向滚动支座。该模型与必做题在节点数、杆件数、几何尺寸、边界条件和荷载方面均不同；也不直接沿用仓库中仅供软件校核的原七杆算例。

1.2 基本假设、坐标与单位

采用理想铰接平面桁架模型：节点与杆件轴线位于同一平面，荷载仅施加于节点；杆件只受轴力，不计弯矩、剪力及扭矩；材料线弹性，截面均匀，计算基于小位移、小变形假设。两题不考虑杆自重、温度应变、初始应变或支座沉降。

整体坐标 x 向右、y 向上，节点位移与节点力沿坐标正方向取正。杆局部轴线由端点 i 指向 j，轴力 N 拉正压负。支座所谓“固定”仅表示对应平动自由度已知，不表示刚架中的转角固定；平面桁架每节点仅设置 ux、uy 两个位移自由度。

表 1.1 统一单位与结果显示约定

物理量	总控及计算单位	报告表格单位
坐标、杆长	m	m
节点位移、杆伸长	m	mm（乘以 1000）
荷载、支反力、轴力	kN	kN
弹性模量 E	kN/m^2	kN/m^2
截面积 A	m^2	m^2
应力	kN/m^2	如转为 MPa 应除以 1000

1.3 计算与复核路线

必做题利用几何对称性组装四阶缩减刚度矩阵，再用一元及二元方程直接求位移、恢复六杆轴力。选做题先用整体平衡和节点法求七杆轴力，再由杆件轴向伸长及几何协调求七个自由位移。机算统一使用 C 程序的直接刚度法及对角缩放 Cholesky 分解。

额外采用独立 Python/numpy 组装与一般线性方程求解核对全部位移、轴力。该复核与 C 程序的矩阵分解实现不同；报告中的手算公式并非仅复制软件输出。所有结果文件由本次实际运行生成，既有仓库的历史校核记录不作为本次验证证据。

2 直接刚度法与程序算法

2.1 单元几何及刚度矩阵

对连接节点 i、j 的第 e 根杆，先由节点坐标求杆长和方向余弦。记 ke=EeAe/Le 为轴向刚度，b 为杆轴向相对位移投影行向量：

$$L_e=\sqrt{\left(x_j-x_i\right)^2+\left(y_j-y_i\right)^2},c_e=\frac{x_j-x_i}{L_e},s_e=\frac{y_j-y_i}{L_e} \tag{2.1}$$

$$b_e=\left[-c_e,-s_e,c_e,s_e\right),k^{(e)}=\frac{E_eA_e}{L_e}b_e^Tb_e \tag{2.2}$$

$$k^{(e)}=\frac{E_eA_e}{L_e}\begin{bmatrix}c_e^2 & c_es_e & -c_e^2 & -c_es_e\\ c_es_e & s_e^2 & -c_es_e & -s_e^2\\ -c_e^2 & -c_es_e & c_e^2 & c_es_e\\ -c_es_e & -s_e^2 & c_es_e & s_e^2\end{bmatrix} \tag{2.3}$$

单元自由度顺序为[uxi,uyi,uxj,uyj]。单根杆刚度矩阵允许刚体运动，因此为半正定矩阵；不能因单元矩阵奇异就判定整个结构无效。是否可求解应以施加实际支座条件后的结构缩减刚度矩阵为准。

2.2 总刚组装与支座处理

内部节点数组采用零基下标。节点 i 的整体自由度位置为 2i、2i+1；把单元四个自由度对应到整体编号，逐项累加单元矩阵。约束方向不进入自由未知量，未约束方向按读入节点次序编号。

$$K=\sum_e T_e^Tk^{(e)}T_e,K_{ff}u_f=F_f-K_{fc}u_c \tag{2.4}$$

其中 T 为自由度提取矩阵，f 为自由方向、c 为约束方向。两题均有 uc=0，因此右端为 Ff。程序仍支持非零支座位移，并准确扣除 Kfcuc；不采用大刚度罚因子或虚设辅助杆。完全固定端点之间的杆保留在模型中，零支座位移时可以输出零轴力。

2.3 方程求解、内力恢复与校核

对稳定结构的 Kff 使用对角缩放，减轻不同刚度量级造成的数值问题。定义 D 的对角项为对应 Kff 对角项的平方根，再对 B 进行 Cholesky 三角分解、前代和回代：

$$D_{qq}=\sqrt{K_{ff,qq}},B=D^{-1}K_{ff}D^{-1}=L\,L^T \tag{2.5}$$

$$B z = D^{-1}\left(F_f - K_{fc} u_c\right), u_f = D^{-1} z \tag{2.6}$$

自由方向对角刚度非正、缩放分解主元不大于 10^{-12} 或数值非有限时，程序拒绝继续计算，避免机构或严重病态模型产生假结果。该主元阈值作用于缩放矩阵。

$$\Delta L_e = c_e\left(u_{xj} - u_{xi}\right) + s_e\left(u_{yj} - u_{yi}\right), N_e = \frac{E_e A_e}{L_e}\Delta L_e \tag{2.7}$$

$$R = K u - F, U = \frac{1}{2} u^T K u = \sum_e \frac{N_e^2 L_e}{2 E_e A_e} \tag{2.8}$$

约束方向 R 为支反力，自由方向为方程残差。除检查残差，还核对所有外力和支反力的水平合力、竖向合力及关于原点的合矩。线性加载条件下，本报告两题满足 $2U = F^{\mathrm{T}}u$；软件输出“load_and_support_work”为最终力与位移的乘积，实际线性加载外力功为其一半。

3 必做题：四节点六杆桁架

3.1 结构数字化与荷载分解

必做题：四节点六杆桁架（按总控信息）

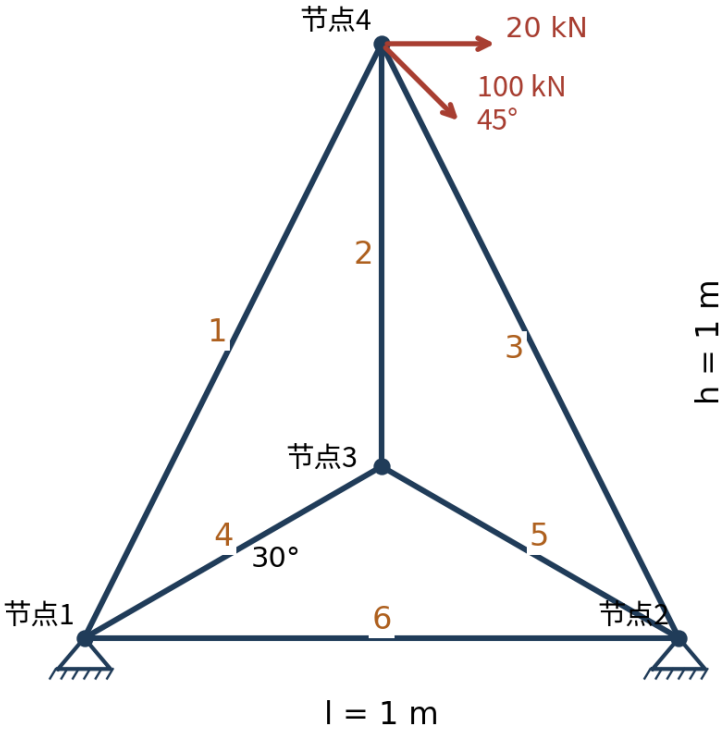


图 3.1 必做题节点、杆件编号及荷载

节点 1、2 的 x、y 方向均约束；节点 3、4 的两方向均自由。杆件编号按原总控文件保留：① 1—4，② 3—4，③ 2—4，④ 1—3，⑤ 3—2，⑥ 1—2。各杆 $E=2\times10^{7}$ kN/m²、$A=0.24$ m²，故 $EA=4.8\times10^{6}$ kN。

表 3.1 必做题节点坐标、约束及荷载

节点	x / m	y / m	约束方向	Fx / kN	Fy / kN
1	0	0	x、y	0	0
2	1	0	x、y	0	0
3	0.5	0.2886751346	无	0	0
4	0.5	1	无	90.7106781187	−70.7106781187

节点 4 的 100 kN 力沿右下 45°，另有 20 kN 向右，按整体坐标分解为：

$$P_x = 20 + \frac{100}{\sqrt{2}} = 90.7106781187\,kN,\quad P_y = -\frac{100}{\sqrt{2}} = -70.7106781187\,kN \tag{3.1}$$

原图内侧两杆与底边夹角为 30°，故节点 3 高度 b=l/(2√3)。节点 4 高度 h 独立输入，不能由图像比例推定。采用已确认的 l=h=1 m 时，竖杆长度 h−b=0.7113248654 m。

3.2 手算：缩减刚度方程与位移

为保留尺寸的一般性，先定义 a=l/2、b=l/(2√3)、d=h−b、Lo=√(a²+h²)，并要求 l>0、h>b。外侧杆①③的方向余弦绝对值为 c=a/Lo、s=h/Lo；内侧杆④⑤的长度 Li=l/√3，方向余弦绝对值为√3/2、1/2。

$$k_o = \frac{EA}{L_o},\quad k_i = \frac{EA}{l/\sqrt{3}},\quad k_v = \frac{EA}{h - l/\left(2\sqrt{3}\right)} \tag{3.2}$$

未知位移顺序取 u=[u3x,u3y,u4x,u4y]ᵀ。内侧两杆在节点 3 的 x-y 耦合项相互抵消，外侧两杆在节点 4 的 x-y 耦合项也相互抵消；竖杆仅耦合节点 3、4 的 y 位移。因此手工组装为：

$$\begin{bmatrix} \frac{3}{2}k_i & 0 & 0 & 0 \\ 0 & \frac{1}{2}k_i + k_v & 0 & -k_v \\ 0 & 0 & 2k_o c^2 & 0 \\ 0 & -k_v & 0 & 2k_o s^2 + k_v \end{bmatrix} \begin{bmatrix} u_{3x} \\ u_{3y} \\ u_{4x} \\ u_{4y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ P_x \\ P_y \end{bmatrix} \tag{3.3}$$

代入 l=h=1 m，得到 ko=4.2932505168×10⁶、ki=8.3138438763×10⁶、kv=6.7479716139×10⁶ kN/m。数值缩减刚度矩阵如下，整体量纲为 kN/m：

$$K_{ff} = 10^6 \begin{bmatrix} 12.4707658145 & 0 & 0 & 0 \\ 0 & 10.9048935520 & 0 & -6.7479716139 \\ 0 & 0 & 1.7173002067 & 0 \\ 0 & -6.7479716139 & 0 & 13.6171724408 \end{bmatrix} \tag{3.4}$$

第一、第三行直接给出 u3x=0、u4x=Px/(2koc²)。第二行给出 u3y=kv·u4y/(ki/2+kv)，代入第四行即得两个竖向位移。为简化表达，定义二阶子矩阵行列式 D：

$$D = \left(k_i/2 + k_v\right)\left(2k_o s^2 + k_v\right) - k_v^2 \tag{3.5}$$

$$u_{3x}=0,\;u_{4x}=\frac{P_x}{2k_oc^2},\;u_{3y}=\frac{k_vP_y}{D},\;u_{4y}=\frac{\left(k_i/2+k_v\right)P_y}{D} \tag{3.6}$$

$$\left[u_{3x},u_{3y},u_{4x},u_{4y}\right)^T=\left[0,-4.6344181862,52.8216777496,-7.4893375207\right)^T\times10^{-6}m \tag{3.7}$$

式中的正负号按 x 向右、y 向上判读：节点 4 水平向右，两自由节点的竖向位移均向下。节点 3 虽受水平非对称外载影响，但荷载未通过竖杆传递水平分量，且内侧两杆对称，因此 u3x 仍为零。

3.3 手算：杆件轴力与支反力

利用杆轴向伸长投影恢复各杆内力；支座节点 1、2 位移均为零。杆⑤的局部方向从 3 指向 2，其 s 为−1/2，但端点相对竖位移为−u3y，故与杆④获得相同轴力。

$$\begin{aligned} N_1&=k_o\left(cu_{4x}+su_{4y}\right), & N_3&=k_o\left(-cu_{4x}+su_{4y}\right),\\ N_2&=k_v\left(u_{4y}-u_{3y}\right), & N_4=N_5&=\frac{k_i}{2}u_{3y},\;N_6=0. \end{aligned} \tag{3.8}$$

代入手算位移得到 N1=+72.6585651998 kN、N2=−19.2649146289 kN、N3=−130.1766773586 kN、N4=N5=−19.2649146289 kN、N6=0。底杆⑥两端均无位移，不发生轴向变形，不能遗漏该杆，但其轴力确为零。

节点 3 平衡给出 N2=N4=N5，节点 4 平衡给出 c(N1−N3)=Px、s(N1+N3)+N2=Py，均与上列数值一致。由两支座节点平衡得到：

$$\begin{aligned} R_{1x}&=-cN_1-\frac{\sqrt{3}}{2}N_4, & R_{1y}&=-sN_1-\frac{1}{2}N_4,\\ R_{2x}&=cN_3+\frac{\sqrt{3}}{2}N_5, & R_{2y}&=-sN_3-\frac{1}{2}N_5. \end{aligned} \tag{3.9}$$

表 3.2 必做题手算支反力

支座	Rx / kN	Ry / kN
1	−15.8099927165	−55.3553390593
2	−74.9006854021	+126.0660171780

竖向反力 R1y 为负表示左支座向下作用，此处支座被建模为可承受双向反力的约束，未引入脱离或单向接触。整体力矩核对：R2y−70.7106781187×0.5−90.7106781187×1=0。

3.4 机算结果及手算对照

表 3.3 必做题全部可动节点位移（mm）

节点/方向	手算	机算	绝对差
3x	0.0000000000	0.0000000000	$<10^{-10}$
3y	-0.0046344182	-0.0046344182	$<10^{-10}$
4x	0.0528216777	0.0528216777	$<10^{-10}$
4y	-0.0074893375	-0.0074893375	$<10^{-10}$

表 3.4 必做题全部杆件轴力（kN）

杆号	连接	手算	机算	拉压
1	1—4	72.658565200	72.658565200	拉
2	3—4	-19.264914629	-19.264914629	压
3	2—4	-130.176677359	-130.176677359	压
4	1—3	-19.264914629	-19.264914629	压
5	3—2	-19.264914629	-19.264914629	压
6	1—2	0.000000000	0.000000000	零

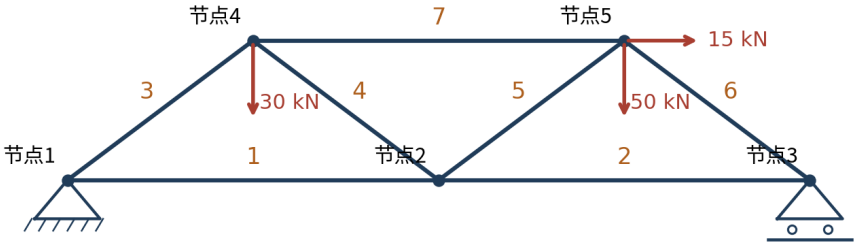
本次程序的自由方程最大残差为 1.4211×10^{-14} kN，整体水平合力为 0，竖向合力及合矩均约 1.4211×10^{-14}。应变能 $U=2.6605331714\times10^{-3}$ kN·m，2U 与最终力-位移乘积一致。上表差值按未舍入数据计算；个位数尾差来自浮点计算和结果文件的输出精度。

在保持 h/l 及荷载、EA 不变的条件下，将所有坐标统一扩大 λ 倍，各轴力不变、各位移扩大 λ 倍。若只改变 h/l，则外侧杆方向改变，轴力分配也随之改变，不可仅进行位移尺度换算。

4 选做题：五节点七杆桁架

4.1 自选模型与数字化

选做题：五节点七杆桁架



跨度 8 m；上节点高度 1.5 m

图 4.1 选做题结构、支座及节点荷载

本题底节点 1、2、3 位于 x=0、4、8 m，上节点 4、5 位于(2,1.5)、(6,1.5) m。节点 1 约束 x、y，节点 3 仅约束 y；其余节点均可动。节点 4 受 30 kN 竖向向下荷载，节点 5 受 15 kN 向右及 50 kN 向下荷载。各杆 $E=2\times10^{7}$ kN/m²、$A=0.01$ m²，$EA=2\times10^{5}$ kN。

表 4.1 选做题节点坐标、约束与荷载

节点	x / m	y / m	约束方向	Fx / kN	Fy / kN
1	0	0	x、y	0	0
2	4	0	无	0	0
3	8	0	y	0	0
4	2	1.5	无	0	-30
5	6	1.5	无	15	-50

表 4.2 选做题杆件几何与连接

杆号	连接	L / m	c	s
1	1—2	4	1	0
2	2—3	4	1	0
3	1—4	2.5	0.8	0.6
4	4—2	2.5	0.8	-0.6
5	2—5	2.5	0.8	0.6
6	5—3	2.5	0.8	-0.6
7	4—5	4	1	0

此题 m+r=7+3=10=2n，计数满足静定条件；结构由两个三角单元及其连接组成，实际几何稳定。计数相等本身不足以证明稳定性，程序还通过缩减刚度正定性检查。本题共有 7 个自由位移，顺序取$[u2x,u2y,u3x,u4x,u4y,u5x,u5y]^{T}$。

4.2 手算：整体平衡与节点法

首先对整体结构取平衡。15 kN 水平力位于 y=1.5 m，对节点 1 产生−22.5 kN·m 力矩，计算支反力时不能遗漏该项：

$$\begin{aligned} \sum F_x &= 0: & R_{1x} + 15 &= 0, \\ \sum M_1 &= 0: & 8R_{3y} - 30\times 2 - 50\times 6 - 15\times 1.5 &= 0, \\ \sum F_y &= 0: & R_{1y} + R_{3y} - 80 &= 0. \end{aligned} \tag{4.1}$$

$$R_{1x} = -15\,kN,\ R_{1y} = 32.1875\,kN,\ R_{3y} = 47.8125\,kN \tag{4.2}$$

假设各杆拉力为正，从节点 1、3 分别列平衡，直接求出两条外斜杆和两条下弦杆：

$$\begin{aligned} 0.6N_3 + R_{1y} &= 0, & N_3 &= -53.6458333333, \\ N_1 + 0.8N_3 + R_{1x} &= 0, & N_1 &= 57.9166666667, \\ 0.6N_6 + R_{3y} &= 0, & N_6 &= -79.6875, \\ -N_2 - 0.8N_6 &= 0, & N_2 &= 63.75. \end{aligned} \tag{4.3}$$

再从节点 4 的竖向及水平平衡求内斜杆④、上弦杆⑦，从节点 2 竖向平衡求内斜杆⑤：

$$\begin{aligned} -0.6N_3 - 0.6N_4 - 30 &= 0, & N_4 &= 3.6458333333, \\ -0.8N_3 + 0.8N_4 + N_7 &= 0, & N_7 &= -45.8333333333, \\ 0.6N_4 + 0.6N_5 &= 0, & N_5 &= -3.6458333333. \end{aligned} \tag{4.4}$$

以上轴力单位均为 kN。作为未使用方程的检查，节点 5 水平平衡为 −0.8N5+0.8N6−N7+15=0，竖向平衡为−0.6N5−0.6N6−50=0；节点 2 水平平衡为 −N1+N2−0.8N4+0.8N5=0，均满足。

4.3 手算：杆伸长与全部自由位移

已知静定桁架的全部轴力后，按胡克定律计算每杆伸长 ΔLe=NeLe/EA。下表采用 mm 显示，正值伸长、负值缩短。

表 4.3 选做题手算杆件伸长

杆号	轴力 / kN	杆长 / m	ΔL / mm
1	57.916666667	4.0	1.1583333333
2	63.750000000	4.0	1.2750000000
3	-53.645833333	2.5	-0.6705729167
4	3.645833333	2.5	0.0455729167
5	-3.645833333	2.5	-0.0455729167
6	-79.687500000	2.5	-0.9960937500
7	-45.833333333	4.0	-0.9166666667

记 δe=ΔLe，以下协调方程统一采用 m。支座条件为 u1x=u1y=u3y=0。逐根杆列轴向投影关系，获得七个独立几何方程：

$$\begin{aligned}
\delta_1 &= u_{2x},\quad \delta_2 = u_{3x} - u_{2x},\\
\delta_3 &= 0.8\,u_{4x} + 0.6\,u_{4y},\\
\delta_4 &= 0.8\left(u_{2x} - u_{4x}\right) - 0.6\left(u_{2y} - u_{4y}\right),\\
\delta_5 &= 0.8\left(u_{5x} - u_{2x}\right) + 0.6\left(u_{5y} - u_{2y}\right).
\end{aligned} \tag{4.5}$$

$$\delta_6 = 0.8\left(u_{3x} - u_{5x}\right) + 0.6\,u_{5y},\quad \delta_7 = u_{5x} - u_{4x} \tag{4.6}$$

杆⑥局部 s=−0.6，而其终点节点 3 的 uy=0，故竖向项为+0.6u5y。先得 u2x=δ1、u3x=δ1+δ2、u5x=u4x+δ7；再由③④⑥分别表示 u4y、u2y、u5y：

$$\begin{aligned}
u_{4y} &= \frac{\delta_3 - 0.8\,u_{4x}}{0.6},\\
u_{2y} &= \frac{\delta_3 + 0.8\,u_{2x} - 1.6\,u_{4x} - \delta_4}{0.6},\\
u_{5y} &= \frac{\delta_6 - 0.8\,u_{3x} + 0.8\,u_{4x} + 0.8\,\delta_7}{0.6}.
\end{aligned} \tag{4.7}$$

把上述表达式代入杆⑤协调方程，可直接消元得到：

$$3.2\,u_{4x}=\delta_5-1.6\,\delta_7+1.6\,u_{2x}-\delta_6+0.8\,u_{3x}+\delta_3-\delta_4 \tag{4.8}$$

代入各杆伸长，右端为 0.0055010416667 m，故 u4x=0.0017190755208 m。再依次回代得 u4y=−0.0034097222222 m、u2y=−0.0042333333333 m、u5x=0.0008024088542 m、u5y=−0.0038347222222 m。加上 u2x=0.0011583333333 m、u3x=0.0024333333333 m，即求得全部七个自由位移。

示例回代：杆③的伸长为 0.8×0.0017190755208+0.6×(−0.0034097222222)=−0.0006705729167 m，与 N3L3/EA 一致。杆⑤也满足 0.8(u5x−u2x)+0.6(u5y−u2y)=−0.0000455729167 m，作为最终协调核对。

4.4 机算刚度方程与结果对照

按前述七自由度顺序组装，机算使用的缩减刚度矩阵如下。矩阵整体乘 10^4，单位为 kN/m；该矩阵与独立单元组装结果一致：

$$K_{ff}=10^4\begin{bmatrix} 20.24 & 0 & -5 & -5.12 & 3.84 & -5.12 & -3.84 \\ 0 & 5.76 & 0 & 3.84 & -2.88 & -3.84 & -2.88 \\ -5 & 0 & 10.12 & 0 & 0 & -5.12 & 3.84 \\ -5.12 & 3.84 & 0 & 15.24 & 0 & -5 & 0 \\ 3.84 & -2.88 & 0 & 0 & 5.76 & 0 & 0 \\ -5.12 & -3.84 & -5.12 & -5 & 0 & 15.24 & 0 \\ -3.84 & -2.88 & 3.84 & 0 & 0 & 0 & 5.76 \end{bmatrix} \tag{4.9}$$

$$F_f=\left[0,0,0,0,-30,15,-50\right]^T kN \tag{4.10}$$

表 4.4 选做题全部可动节点自由位移（mm）

节点/方向	手算	机算	绝对差
2x	1.1583333333	1.1583333333	$<10^{-10}$
2y	-4.2333333333	-4.2333333333	$<10^{-10}$
3x	2.4333333333	2.4333333333	$<10^{-10}$
4x	1.7190755208	1.7190755208	$<10^{-10}$
4y	-3.4097222222	-3.4097222222	$<10^{-10}$
5x	0.8024088542	0.8024088542	$<10^{-10}$
5y	-3.8347222222	-3.8347222222	$<10^{-10}$

节点 3 是仅竖向受约束的滚动支座，其水平位移必须计入可动节点结果；u3y=0 为已知边界条件。节点 1 的两个位移均为零。

表 4.5 选做题全部杆件轴力（kN）

杆号	连接	手算	机算	拉压
1	1—2	57.916666667	57.916666667	拉
2	2—3	63.750000000	63.750000000	拉
3	1—4	-53.645833333	-53.645833333	压
4	4—2	3.645833333	3.645833333	拉
5	2—5	-3.645833333	-3.645833333	压
6	5—3	-79.687500000	-79.687500000	压
7	4—5	-45.833333333	-45.833333333	压

机算支反力为 R1x=−15 kN、R1y=32.1875 kN、R3y=47.8125 kN，与手算完全一致。自由方程最大残差为 5.6843×10^{-14} kN；整体合力及合矩误差量级为 10^{-13}。应变能 U=0.1530319552951 kN·m，最终荷载乘积为 $-30u4y+15u5x-50u5y=0.3060639105903$ kN·m，满足 $2U=F^{T}u$。

5 程序功能、使用与计算结论

5.1 功能与数据组织

本交付采用 C11 命令行计算核心，输入总控信息即可完成两题；不依赖图形界面或 Python 环境。基于原仓库 truss.c 仅补充注释，未改变计算逻辑。附录 A 完整收录本次交付的 truss.c，说明数据结构、编号映射、输入检查、刚度组装、边界处理、缩放分解、轴力恢复及校核等。

表 5.1 主要函数及职责

函数	用途
token / key / number / integer	读取关键词、数值和注释，检查格式
read_model / prepare	读取模型、映射外部编号、求几何并检查
required_model / write_model	按 l、h 创建必做题及导出总控
cholesky	原地分解、前代与回代
solve	组装、边界处理、求解、恢复结果及校核
main / wmain	命令行参数、文件管理及退出状态

节点号、杆号无需连续，但必须唯一；连接端点必须存在。程序支持每杆独立 E、A、单向支座及非零支座位移。输入中的#后为注释。支持 UTF-8 BOM，数值使用英文小数点及科学计数法。节点数量范围 2～1000、杆数 1～10000；这些是输入容量限制，不等于大型模型的性能保证。

5.2 运行方式与输出内容

Windows x64 可执行程序 truss.exe 来自用户上传仓库，复制后核对 SHA-256 保持一致；本次在 Linux 中运行新编译的 truss_linux，未对 Windows 二进制进行实际运行验证。两个版本使用相同核心算法；Windows 使用宽字符路径入口，支持中文文件路径。

Windows 中在“程序”目录双击“运行两题.bat”，会依次读入两份总控并更新对应机算结果。Linux 中运行 bash 程序/运行两题.sh。若单独执行，命令格式为：

truss.exe input.txt output.txt

./truss_linux input.txt output.txt

输出包括自由度映射、缩减刚度和修正荷载、杆长与方向余弦、节点位移、各杆轴力/伸长/应力、支反力及残差/整体平衡/能量检查。输入或求解失败返回非零退出码，不能将部分输出误当作成功结果。详细路径与编译指令见交付包“使用说明.txt”。

5.3 实际校核与软件特点

表 5.2 本次实际执行的校核

检查项	结果与证据
C11 严格编译	gcc -O2 -Wall -Wextra -Werror，成功无警告
两题计算与复核	全部自由位移与全部杆轴力符合解析手算和独立矩阵解
重复运行	两题结果文件与本次首轮输出逐字节一致
非零支座位移	单杆验证 u20x=0.0011 m、N100=10 kN
非法模型拒绝	零长度杆、非正 E、自由横向机构均报错，退出码 2
Windows 可执行文件	与仓库文件校验值一致；本次未运行 Windows 版本

与只输出杆力的基础实现相比，本程序同时输出节点位移、支反力、几何量及数值校核，便于定位单位、符号和约束错误。通用自由度表适用于任意节点编号和单向支座；缩减方程准确处理已知位移；对角缩放及主元检查能拒绝不稳定模型。完整源码采用标准 C 及数学库，便于课堂理解和移植。

当前实现使用稠密矩阵，总刚存储量约 $O((2n)^2)$，分解量约 $O(nf^3)$，适合课程小规模桁架。它不计算刚架弯矩、材料非线性、大变形、屈曲、杆件承载力或支座接触。数值平衡检查通过说明线性模型求解正确，不能替代真实结构模型合理性及强度稳定性验算。

5.4 计算结论

必做题节点 4 水平位移最大，为+0.0528216777 mm；两自由节点竖向均向下。最大拉力出现在杆①，为 72.6585651998 kN；最大压力绝对值出现在杆③，为 130.1766773586 kN；底杆⑥轴力为零。

选做题最大自由竖位移绝对值出现在节点 2，为 4.2333333333 mm；最大水平位移出现在滚动支座节点 3，为 2.4333333333 mm。杆②拉力最大，为 63.75 kN；杆⑥压力绝对值最大，为 79.6875 kN。两题手算与机算的全部必需结果一致，整体平衡、节点平衡、杆件协调及能量核对通过。

附录 A 程序全文和程序注释

以下为交付文件“程序/truss.c”的完整文本，共 405 行。程序在用户上传的计算核心基础上增加解释性注释，计算语句未改动。其独立编译和运行无需本报告制作或复核使用的 Python 工具。

```
/* 平面桁架直接刚度法，单位：力 kN，长度/位移 m，E kN/m^2，A m^2。
 * 假设：小变形、线弹性、理想铰接，杆件仅传递轴力；拉力为正。
 * 输入：TRUSS2D 1 / NODES n / 节点数据 / ELEMENTS m / 单元数据 / END。
 * 节点：编号 x y 固定 x 固定 y 指定位移 x 指定位移 y 荷载 x 荷载 y。
 * 单元：编号 起点编号 终点编号 E A；# 开头的内容为行注释。
 * 编号无需连续；每节点有全局 x、y 两个自由度，内部按读入次序编号。
 */
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <ctype.h>
#include <errno.h>
#include <limits.h>

/* Windows 命令行保留完整 Unicode 文件路径；其他平台继续使用标准 C 入口。 */
#ifdef _WIN32
#include <wchar.h>
typedef wchar_t PathChar;
#define PATH_TEXT(text) L##text
#define path_compare wcscmp
#define path_number wcstod
#define path_open _wfopen
#define PROGRAM_ENTRY wmain
#else
typedef char PathChar;
```

```
#define PATH_TEXT(text) text
#define path_compare strcmp
#define path_number strtod
#define path_open fopen
#define PROGRAM_ENTRY main
#endif

/* 节点数据结构：id 是文件中的外部编号；x/y 为坐标。
 * fix[0]/fix[1] 分别表示 x/y 方向是否约束，1 约束、0 自由。
 * p[0]/p[1] 是已知支座位移，不是荷载；f[0]/f[1] 是节点外力。
 * 数组方向约定固定为 0=x、1=y，避免位移、荷载方向混淆。 */
typedef struct { int id, fix[2]; double x, y, p[2], f[2]; } Node;
/* 杆件数据：i/j 是节点数组下标，不是外部节点号；read_model 完成转换。
 * E 为弹性模量，A 为截面积，L 为实际杆长，c/s 为方向余弦。
 * 原始编号 id 仅用于识别及输出；改变杆端次序不改变拉正轴力。 */
typedef struct { int id, i, j; double E, A, L, c, s; } Element;
typedef struct { int n, m; Node *node; Element *elem; } Model;
static int input_line = 1;

/* 所有异常都给出明确原因并返回非零退出码，避免输出看似正常的假结果。 */
static _Noreturn void die(const char *message) {
    fprintf(stderr, "ERROR (input line %d): %s\n", input_line, message);
    exit(2);
}
/* 统一采用 calloc：将新内存置零，使矩阵对号累加从零开始。
 * count=0 时仍分配一个对象，允许完全约束模型的空自由方程。
 * 分配失败立即终止，禁止对空指针继续计算。调用者负责 free。 */
static void *allocate(size_t count, size_t size) {
    void *p = calloc(count ? count : 1, size);
    if (!p) die("Insufficient memory.");
    return p;
}
/* 逐词读取并跳过 # 注释；只接收 ASCII 关键词，数值用点号作小数点。 */
```

```
/* 输入词法扫描：逐字符略过空白和 # 注释，维护报错行号。
 * token 缓冲区由调用者提供，容量必须至少 128 字节。
 * 最多接收 127 字节，再写入字符串终止符；EOF 返回 0。
 * 用 unsigned char 转换后调用 isspace，避免负 char 引发未定义行为。 */
static int token(FILE *f, char *buf) {
   int ch, len = 0;
   for (;;) {
      ch = fgetc(f);
      if (ch == EOF) return 0;
      if (ch == '#') { while (ch != '\n' && ch != EOF) ch = fgetc(f); }
      if (ch == '\n') ++input_line;
      if (ch != EOF && !isspace((unsigned char)ch)) break;
   }
   do {
      if (len >= 127) die("Input token is too long.");
      buf[len++] = (char)ch;
      ch = fgetc(f);
   } while (ch != EOF && !isspace((unsigned char)ch) && ch != '#');
   if (ch == '#') { while (ch != '\n' && ch != EOF) ch = fgetc(f); }
   if (ch == '\n') ++input_line;
   buf[len] = 0;
   return 1;
}
/* 关键词大小写敏感；按既定数据顺序检查，发现错位立即报错。 */
static void key(FILE *f, const char *expected) {
   char b[128];
   if (!token(f, b) || strcmp(b, expected)) die("Missing or unexpected
keyword.");
}
/* strtod 接受普通小数和科学计数法；end 检查整词被消费。
 * errno 拒绝数值转换溢出；isfinite 拒绝 NaN 和无穷大。
 * 本程序使用默认 C 数值区域设置，小数点为英文句点。 */
static double number(FILE *f) {
```

```
    char b[128], *end; double value;
    if (!token(f, b)) die("Missing numeric value.");
    errno = 0; value = strtod(b, &end);
    if (errno || *end || !isfinite(value)) die("Invalid or non-finite number.");
    return value;
}
/* 整数字段先按 double 解析，再检查 int 范围和无小数部分。
 * 节点数量、杆件数量、编号及约束标志均使用此函数。 */
static int integer(FILE *f) {
    double v = number(f);
    if (v < INT_MIN || v > INT_MAX || floor(v) != v) die("Expected integer.");
    return (int)v;
}
/* 将输入的外部编号映射为零基数组下标；线性检索便于课程理解。
 * 未找到节点表示连接信息错误，不可把编号直接作为数组下标。 */
static int find_node(const Model *a, int id) {
    int i;
    for (i = 0; i < a->n; ++i) if (a->node[i].id == id) return i;
    die("An element references a missing node.");
}
/* 零长度杆及非正材料参数不符合本线弹性桁架模型，必须在组装前排除。 */
static void prepare(Model *a) {
    int e;
    for (e = 0; e < a->m; ++e) {
        Element *b = a->elem + e;
        double dx = a->node[b->j].x - a->node[b->i].x;
        double dy = a->node[b->j].y - a->node[b->i].y;
        b->L = hypot(dx, dy);
        if (!(b->L > 0) || !isfinite(b->L) || b->i == b->j)
            die("Element length must be finite and positive.");
        if (!(b->E > 0) || !(b->A > 0)) die("E and A must be positive.");
        b->c = dx / b->L; b->s = dy / b->L;
        if (!isfinite(b->E * b->A / b->L)) die("Element stiffness overflow.");
```

```
    }
}
/* 完整读入流程：版本 -> 节点表 -> 杆件表 -> END -> 几何预处理。
 * 节点坐标、支座条件和荷载保存在同一行；每根杆独立给 E、A。
 * 重复编号、缺失端点、自由方向指定位移、END 后多余词均被拒绝。
 * 仅关键词和数值必须为 ASCII；# 注释可写 UTF-8 中文。 */
static void read_model(FILE *f, Model *a) {
    int i, j, first = fgetc(f); char b[128];
    /* 兼容 Windows 记事本写出的 UTF-8 BOM；其余内容仍用 ASCII 字段。 */
    if (first == 0xEF) {
        if (fgetc(f) != 0xBB || fgetc(f) != 0xBF) die("Invalid UTF-8 BOM.");
    } else if (first != EOF) ungetc(first,f);
    key(f, "TRUSS2D"); if (integer(f) != 1) die("Unsupported format version.");
    key(f, "NODES"); a->n = integer(f);
    if (a->n < 2 || a->n > 1000) die("Node count must be 2..1000.");
    a->node = allocate((size_t)a->n, sizeof(Node));
    for (i = 0; i < a->n; ++i) {
        Node *n = a->node + i;
        n->id = integer(f); n->x = number(f); n->y = number(f);
        for (j = 0; j < i; ++j)
            if (a->node[j].id == n->id) die("Duplicate node ID.");
        n->fix[0] = integer(f); n->fix[1] = integer(f);
        n->p[0] = number(f); n->p[1] = number(f);
        n->f[0] = number(f); n->f[1] = number(f);
        for (j = 0; j < 2; ++j) {
            if (n->fix[j] != 0 && n->fix[j] != 1) die("Fixity must be 0 or 1.");
            if (!n->fix[j] && n->p[j] != 0) die("Free DOF cannot have prescribed
displacement.");
        }
    }
    key(f, "ELEMENTS"); a->m = integer(f);
    if (a->m < 1 || a->m > 10000) die("Element count must be 1..10000.");
    a->elem = allocate((size_t)a->m, sizeof(Element));
```

```
    for (i = 0; i < a->m; ++i) {
        Element *e = a->elem + i;
        e->id = integer(f); e->i = find_node(a, integer(f));
        e->j = find_node(a, integer(f)); e->E = number(f); e->A = number(f);
        for (j = 0; j < i; ++j)
            if (a->elem[j].id == e->id) die("Duplicate element ID.");
    }
    key(f, "END"); if (token(f, b)) die("Unexpected data after END.");
    if (ferror(f)) die("Input read failed.");
    prepare(a);
}
/* 必做题：底杆 6 长 l，顶点 4 的高度为 h；节点 3 由两侧 30 度确定。
 * 节点 1、2 的两个方向均固定；100 kN 向右下 45 度，另有向右 20 kN。
 * 图中没有指定 l、h 数值；输入值均为用户参数，不能当作题图已知尺寸。 */
static void required_model(Model *a, double l, double h) {
    const int ends[6][2] = {{0,3},{2,3},{1,3},{0,2},{2,1},{0,1}};
    int i;
    if (!(l > 0) || !isfinite(l)) die("l must be finite and positive (m).");
    if (!(h > l/(2*sqrt(3.0))) || !isfinite(h))
        die("h must be finite and above node 3: h > l/(2*sqrt(3)) (m).");
    a->n = 4; a->m = 6;
    a->node = allocate(4, sizeof(Node)); a->elem = allocate(6, sizeof(Element));
    for (i = 0; i < 4; ++i) a->node[i].id = i + 1;
    a->node[1].x = l;
    a->node[2].x = l / 2; a->node[2].y = l / (2 * sqrt(3.0));
    a->node[3].x = l / 2; a->node[3].y = h;
    for (i = 0; i < 2; ++i) a->node[i].fix[0] = a->node[i].fix[1] = 1;
    a->node[3].f[0] = 20 + 100 / sqrt(2.0); a->node[3].f[1] = -100 / sqrt(2.0);
    for (i = 0; i < 6; ++i) {
        Element *e = a->elem + i;
        e->id = i + 1; e->i = ends[i][0]; e->j = ends[i][1];
        e->E = 2e7; e->A = 0.24;
    }
```

```
    prepare(a);
}
/* 总控输出采用 17 位有效数字，尽量保证 double 读写往返精度。
 * 输出端点使用外部编号，不能把内部 i/j 下标直接写回文件。
 * 本函数只写文件；输出错误在主函数统一检查。 */
static void write_model(FILE *f, const Model *a, double l, double h) {
    int i;
    fprintf(f, "# Required problem: base length l = %.17g m; top height h = %.17g
m.\n", l,h);
    fprintf(f, "# This numerical parameter is supplied by the user, not by the
diagram.\n");
    fprintf(f, "TRUSS2D 1\nNODES %d\n# id x y fix_x fix_y u_x u_y F_x F_y\n", a-
>n);
    for (i = 0; i < a->n; ++i) {
        const Node *n = a->node + i;
        fprintf(f, "%d %.17g %.17g %d %d %.17g %.17g %.17g %.17g\n",
            n->id,n->x,n->y,n->fix[0],n->fix[1],n->p[0],n->p[1],n->f[0],n->f[1]);
    }
    fprintf(f, "ELEMENTS %d\n# id node_i node_j E A\n", a->m);
    for (i = 0; i < a->m; ++i) {
        const Element *e = a->elem + i;
        fprintf(f, "%d %d %d %.17g %.17g\n", e->id,a->node[e->i].id,
            a->node[e->j].id,e->E,e->A);
    }
    fprintf(f, "END\n");
}
/* 对角缩放后的 Cholesky：B = D^-1 Kff D^-1 = LL^T。
 * 只求解自由自由度，约束位移的作用移到右端项；无需罚因子。
 * 负/近零主元意味着机构、约束不足或病态，不能继续计算。 */
/* b 用行优先 n*n 数组保存缩放刚度；下三角原地改写为 L。
 * i/j/k 的三重循环对应 Lij=(Bij-sum(Lik*Ljk))/Ljj。
 * 对角项取平方根前检查 >1e-12；此阈值作用于缩放后的矩阵。
 * 前代求 L*y=缩放荷载；回代求 L^T*x=y，结果原地保存在 x。
```

```
 * 上三角旧数据无需更新，回代明确读取下三角的转置位置。
 * n=0 时循环均跳过，支持没有自由未知量的完全约束模型。 */
static void cholesky(double *b, double *x, int n) {
   int i, j, k;
   for (i = 0; i < n; ++i) for (j = 0; j <= i; ++j) {
      double v = b[(size_t)i*n+j];
      for (k = 0; k < j; ++k) v -= b[(size_t)i*n+k]*b[(size_t)j*n+k];
      if (i == j) {
         if (!(v > 1e-12) || !isfinite(v))
            die("Unstable or ill-conditioned structure: stiffness is not positive
definite.");
         b[(size_t)i*n+j] = sqrt(v);
      } else b[(size_t)i*n+j] = v / b[(size_t)j*n+j];
   }
   for (i = 0; i < n; ++i) {
      for (j = 0; j < i; ++j) x[i] -= b[(size_t)i*n+j]*x[j];
      x[i] /= b[(size_t)i*n+i];
   }
   for (i = n-1; i >= 0; --i) {
      for (j = i+1; j < n; ++j) x[i] -= b[(size_t)j*n+i]*x[j];
      x[i] /= b[(size_t)i*n+i];
   }
}
/* 求解器主流程及变量：
 * nd=2*n 为完整自由度数；nf 为未约束自由度数。
 * 完整编号 2*i 是节点 i 的 x，2*i+1 是 y（均为零基）。
 * map[q] 将自由方程 q 映射到完整自由度；仅含未约束方向。
 * K 为完整刚度，u 为完整位移，r=K*u-F 为反力/自由残差。
 * B 为缩放缩减刚度，rhs 为缩放右端，scale[q]=sqrt(Kqq)。
 * 原始 K 不被分解覆盖，保留用于反力和残差核对。
 * 当前实现是稠密矩阵：内存约 O(nd^2)，分解约 O(nf^3)。 */
static void solve(const Model *a, FILE *out, double l, double h) {
   int nd = 2*a->n, nf = 0, i, j, e;
```

```
    int *map = allocate((size_t)nd, sizeof(int));
    /* K 对应课件总刚数组 r；node.f/u 对应 dp 求解前的荷载/求解后的位移。
     * 输出 N 对应课件杆力数组 h，注意与本程序的顶高参数 h 区别。 */
    double *K = allocate((size_t)nd*nd, sizeof(double));
    double *u = allocate((size_t)nd, sizeof(double));
    double *r = allocate((size_t)nd, sizeof(double));
    double *B, *rhs, *scale, residual = 0, force_scale = 1;
    double energy = 0, work = 0, balance[3] = {0,0,0};
    /* 课件 elcs 几何变换：q=(-c,-s,c,s)，ke=(EA/L)q q^T。
     * 课件 i0、j0 为杆端自由度起址；这里使用完整全局起址 2*i、2*j。
     * 四块分别为 +C、-C、-C、+C；两端均检查约束，固定点无需排在最前。 */
    for (e = 0; e < a->m; ++e) {
        const Element *b = a->elem + e;
        int d[4] = {2*b->i,2*b->i+1,2*b->j,2*b->j+1};
        double q[4] = {-b->c,-b->s,b->c,b->s}, k = b->E*b->A/b->L;
        for (i = 0; i < 4; ++i) for (j = 0; j < 4; ++j)
            K[(size_t)d[i]*nd+d[j]] += k*q[i]*q[j];
    }
    /* 先回填约束位移；自由位移先置零，再建立 map。
     * 节点可任意编号与排序，单向滚动支座也可直接表示。 */
    for (i = 0; i < nd; ++i) {
        const Node *n = a->node + i/2;
        u[i] = n->fix[i%2] ? n->p[i%2] : 0;
        if (!n->fix[i%2]) map[nf++] = i;
    }
    /* 施加边界条件：Kff*uf=Ff-Kfc*uc。
     * u 的自由方向目前为零，所以遍历完整列计算 Kfc*uc。
     * 对角缩放 Dqq=sqrt(Kqq)，B=D^-1*Kff*D^-1；
     * 求解 B*z=D^-1*右端，最后 uf=D^-1*z。 */
    B = allocate((size_t)nf*nf, sizeof(double));
    rhs = allocate((size_t)nf, sizeof(double)); scale = allocate((size_t)nf,
sizeof(double));
    for (i = 0; i < nf; ++i) {
```

```
        double diag = K[(size_t)map[i]*nd+map[i]];
        if (!(diag > 0) || !isfinite(diag)) die("A free DOF has zero or invalid
stiffness.");
        scale[i] = sqrt(diag);
        rhs[i] = a->node[map[i]/2].f[map[i]%2];
        for (j = 0; j < nd; ++j) rhs[i] -= K[(size_t)map[i]*nd+j]*u[j];
        rhs[i] /= scale[i];
    }
    for (i = 0; i < nf; ++i) for (j = 0; j < nf; ++j)
        B[(size_t)i*nf+j] = K[(size_t)map[i]*nd+map[j]] / scale[i] / scale[j];
    cholesky(B, rhs, nf);
    for (i = 0; i < nf; ++i) u[map[i]] = rhs[i]/scale[i];
    /* 用原始 K 回代，约束自由度得到反力，自由自由度得到残差。 */
    for (i = 0; i < nd; ++i) {
        const Node *n = a->node + i/2; double ku = 0;
        if (!isfinite(u[i])) die("Displacement overflow.");
        for (j = 0; j < nd; ++j) ku += K[(size_t)i*nd+j]*u[j];
        r[i] = ku - n->f[i%2]; energy += 0.5*u[i]*ku;
        force_scale = fmax(force_scale, fmax(fabs(ku), fabs(n->f[i%2])));
        if (!n->fix[i%2]) residual = fmax(residual, fabs(r[i]));
        work += u[i] * (n->f[i%2] + (n->fix[i%2] ? r[i] : 0));
    }
    /* 输出使用 12 位小数的科学计数格式，避免微小位移被四舍五入为零。
     * 小模型附原始缩减刚度，便于与手算对照；不输出分解后的矩阵。 */
    fprintf(out, "PLANE TRUSS - DIRECT STIFFNESS METHOD\n");
    fprintf(out, "Units: m, kN, kN/m^2. Axial force: tension +, compression -.\n");
    fprintf(out, "Nodes=%d Elements=%d Free_DOF=%d\n",a->n,a->m,nf);
    if (l > 0) fprintf(out, "Required problem: base l=%.12g m, top h=%.12g m,
h/l=%.12g (user parameters).\n",l,h,h/l);
    fprintf(out, "\nFREE DOF MAP\nequation node_id direction\n");
    for (i = 0; i < nf; ++i)
        fprintf(out, "%d %d %c\n",i+1,a->node[map[i]/2].id,map[i]%2 ? 'Y' : 'X');
    /* 教学用小模型打印未三角化的原始 Kff 和修正荷载 Ff-Kfc*uc。 */
```

```
    if (nf <= 20) {
        fprintf(out, "\nREDUCED STIFFNESS (kN/m) AND RHS (kN)\n");
        fprintf(out, "equation [Kff row] | corrected_rhs\n");
        for (i = 0; i < nf; ++i) {
            double f = a->node[map[i]/2].f[map[i]%2];
            fprintf(out, "%d",i+1);
            for (j = 0; j < nf; ++j) fprintf(out," %.12e",K[(size_t)map[i]*nd+map[j]]);
            for (j = 0; j < nd; ++j) if (a->node[j/2].fix[j%2])
                f -= K[(size_t)map[i]*nd+j]*a->node[j/2].p[j%2];
            fprintf(out, " | %.12e\n",f);
        }
    } else fprintf(out,"\nKff matrix display omitted for more than 20 free DOFs.
\n");
    fprintf(out, "\nELEMENT GEOMETRY\nid L(m) cos(theta) sin(theta)
EA/L(kN/m)\n");
    for (e = 0; e < a->m; ++e) {
        const Element *b = a->elem + e;
        fprintf(out,"%d %.12e %.12e %.12e %.12e\n",b->id,b->L,b->c,b->s,b->E*b-
>A/b->L);
    }
    fprintf(out, "\nNODE RESULTS\nid ux(m) uy(m) Rx(kN) Ry(kN) fix_x fix_y\n");
    for (i = 0; i < a->n; ++i) {
        const Node *n = a->node + i;
        double rx = n->fix[0] ? r[2*i] : 0, ry = n->fix[1] ? r[2*i+1] : 0;
        fprintf(out, "%d %.12e %.12e %.12e %.12e %d %d\n",
            n->id,u[2*i],u[2*i+1],rx,ry,n->fix[0],n->fix[1]);
        balance[0] += rx+n->f[0]; balance[1] += ry+n->f[1];
        balance[2] += n->x*(ry+n->f[1])-n->y*(rx+n->f[0]);
    }
    if (l > 0) {
        fprintf(out, "\nPARAMETER COEFFICIENTS\nid ux/l uy/l (dimensionless)
\n");
        for (i = 0; i < a->n; ++i)
```

```
        fprintf(out,"%d %.12e %.12e\n",a->node[i].id,u[2*i]/l,u[2*i+1]/l);
    }
    /* 轴向伸长是杆端相对位移沿局部轴线的投影。
     * N=(EA/L)*伸长：伸长为正时拉力为正，缩短时压力为负。
     * 应力 N/A 输出为 kN/m^2；换为 MPa 时应再除以 1000。 */
    fprintf(out, "\nELEMENT RESULTS\nid node_i node_j L(m) N(kN)
extension(m) stress(kN/m^2)\n");
    for (e = 0; e < a->m; ++e) {
        const Element *b = a->elem + e;
        double ext = b->c*(u[2*b->j]-u[2*b->i])+b->s*(u[2*b->j+1]-u[2*b->i+1]);
        double N = b->E*b->A/b->L*ext;
        fprintf(out, "%d %d %d %.12e %.12e %.12e %.12e\n",
            b->id,a->node[b->i].id,a->node[b->j].id,b->L,N,ext,N/b->A);
    }
    /* 校核含自由方程残差、整体合力与合矩、能量恒等式。
     * 能量 U=0.5*u^T*K*u；右端 work=u^T*(F+支反力)。
     * work 是最终力与位移乘积，不是线性加载过程的实际外力功。
     * 比较 2*U=work；有支座位移时不可漏掉支反力对应的项。
     * balance 只使用约束方向反力与外力，不将自由残差算作反力。 */
    fprintf(out,
"\nCHECKS\nmax_free_residual(kN)=%.12e\nrelative_free_residual=%.12e\n",
        residual,residual/force_scale);
    fprintf(out,
"sum_Fx(kN)=%.12e\nsum_Fy(kN)=%.12e\nsum_Mz(kN*m)=%.12e\n",
        balance[0],balance[1],balance[2]);
    fprintf(out,
"strain_energy(kN*m)=%.12e\nload_and_support_work(kN*m)=%.12e\n",
        energy,work);
    fprintf(out, "energy_identity_error(kN*m)=%.12e\n",2*energy-work);
    if (!isfinite(energy) || residual/force_scale > 1e-8) die("Numerical verification
failed.");
    free(map); free(K); free(u); free(r); free(B); free(rhs); free(scale);
}
```

```
/* 命令行入口：
 * 普通模式：truss 输入总控.txt [输出结果.txt]。
 * 参数模式：truss --required l h [输出结果.txt]。
 * 导出模式：truss --write-required l h 输出总控.txt。
 * 无输出路径时结果写 stdout；Windows 使用 wmain 和 _wfopen
 * 保留中文路径，Linux 使用 main 和 fopen。
 * 成功返回 0；用法错误返回 1；输入/求解错误经 die 返回 2。
 * 主函数负责关闭文件并释放模型；异常时由进程退出回收内存。 */
int PROGRAM_ENTRY(int argc, PathChar **argv) {
  Model a = {0}; FILE *in, *out = stdout; double l = 0, h = 0; int write_only = 0;
  const PathChar *output = NULL;
  if (argc >= 2 && (!path_compare(argv[1],PATH_TEXT("--required")) || !
path_compare(argv[1],PATH_TEXT("--write-required")))) {
    PathChar *end; write_only = !path_compare(argv[1],PATH_TEXT("--write-
required"));
    if (argc < 4 || argc > 5 || (write_only && argc != 5)) die("Incorrect
parameter count: l_m and h_m are both required.");
    errno=0; l=path_number(argv[2],&end);
    if (errno || *end) die("Invalid l parameter.");
    errno=0; h=path_number(argv[3],&end); if (errno || *end) die("Invalid h
parameter.");
    required_model(&a,l,h); if (argc == 5) output=argv[4];
  } else {
    if (argc < 2 || argc > 3) {
      fprintf(stderr,"Usage: truss.exe input.txt [output.txt]\n"
        "       truss.exe --required l_m h_m [output.txt]\n"
        "       truss.exe --write-required l_m h_m input.txt\n"); return 1;
    }
    in=path_open(argv[1],PATH_TEXT("r")); if (!in) die("Cannot open input
file.");
    read_model(in,&a); fclose(in); if (argc == 3) output=argv[2];
  }
```

```
    if (output) { out=path_open(output,PATH_TEXT("w")); if (!out) die("Cannot
open output file."); }
    if (write_only) write_model(out,&a,l,h); else solve(&a,out,l,h);
    if (ferror(out)) die("Output write failed.");
    if (out != stdout && fclose(out)) die("Output close failed.");
    free(a.node); free(a.elem); return 0;
}
```

附录 B 必做题与选做题总控信息

以下为两份独立总控文本的完整内容。格式由现有程序定义；每节点顺序为编号、x、y、约束 x、约束 y、已知 ux、已知 uy、Fx、Fy；每杆顺序为编号、起点编号、终点编号、E、A。数值单位为 kN、m。

B.1 必做题总控信息

```
# Mandatory six-bar truss. Units: m, kN.
# Supplemental report assumption: l=1 m, h=1 m; original diagram has no dimensions.
# E=2e7 kN/m^2; A=0.24 m^2; tension positive.
TRUSS2D 1
NODES 4
# id x y fix_x fix_y u_x u_y F_x F_y
1 0 0 1 1 0 0 0 0
2 1 0 1 1 0 0 0 0
3 0.5 0.28867513459481292 0 0 0 0 0 0
4 0.5 1 0 0 0 0 90.710678118654741 -70.710678118654741
ELEMENTS 6
# id node_i node_j E A
1 1 4 20000000 0.23999999999999999
2 3 4 20000000 0.23999999999999999
3 2 4 20000000 0.23999999999999999
4 1 3 20000000 0.23999999999999999
5 3 2 20000000 0.23999999999999999
6 1 2 20000000 0.23999999999999999
END
```

B.2 选做题总控信息

```
# Optional problem: five-node seven-bar determinate truss, selected for this report.
# Units: m, kN, E in kN/m^2, A in m^2. Tension positive.
# Span=8 m; top nodes height=1.5 m; E=2e7; A=0.01.
```

```
TRUSS2D 1
NODES 5
# id x y fix_x fix_y prescribed_ux prescribed_uy Fx Fy
1 0 0 1 1 0 0 0 0
2 4 0 0 0 0 0 0 0
3 8 0 0 1 0 0 0 0
4 2 1.5 0 0 0 0 0 -30
5 6 1.5 0 0 0 0 15 -50
ELEMENTS 7
# id node_i node_j E A
1 1 2 20000000 0.01
2 2 3 20000000 0.01
3 1 4 20000000 0.01
4 4 2 20000000 0.01
5 2 5 20000000 0.01
6 5 3 20000000 0.01
7 4 5 20000000 0.01
END
```